

GMP3 OPTİMİZASYONU

Versiyon	Açıklama	Tarih	Yazar
1.0	GMP3 Optimizasyon Dokümanı Hazırlandı	15-01-2026	Ali Yusuf Akbay
1.1	UniqueID ile ilgili kısımlar eklendi	15-05-2026	Ali Yusuf Akbay

Bu doküman, GMP3 entegrasyonunuzu daha optimize şekilde geliştirmenize yardımcı olmayı ve entegrasyon sırasında performans düşüşüne yol açabilecek yaygın hataları ele almayı amaçlamaktadır.

İÇİNDEKİLER

LogThreadOpen	2
IsTcpKeepAlive	2
MultipleCommand (BATCH MODE)	3
Ethernet VS USB	4
Banka Host IP'leri	4
Hafifletilmiş Sorgular	4
Unique ID ile ilgili öneri	5
Gereksiz FP3_GetTicket Kullanımı	5
Entegrasyon Yazılımında Geçen Süre	6
Güncel Versiyon Kullanımı	7
Dijital İşyeri Nüshası	8
Müşteri Nüshası ile İşyeri Nüshası Ardında Geçen Süre	8

LogThreadOpen

GMP.XML dosyasında yer alan log ayarlarında **LogThreadOpen** anahtarının **TRUE** olarak ayarlanmasını önemle tavsiye ederiz.

LogThreadOpen değerinin **FALSE** olması durumunda, log yazma işlemleri önceliklendirildiğinden işlem sürelerinde yavaşlama yaşanabilir. Uygulamanız beklenen performansı sağlamıyorsa, LogThreadOpen değerini **TRUE** olarak ayarlayarak tekrar denemeniz önerilir. Bu nedenle, her zaman bu ayarın **TRUE** olması tavsiye edilmektedir.

IsTcpKeepAlive

GMP.XML dosyasının **Interface** bölümünde yer alan bu anahtar, **DLL 16v25** sürümü ile birlikte eklenmiştir. Anahtarın kullanılabilmesi için cihazın **Ethernet bağlantısı** üzerinden haberleşmesi gerekmektedir.

Değer **TRUE** olarak ayarlandığında, cihaz ile yazılım arasındaki bağlantı periyodik olarak (**AckTimeOut** süresi baz alınarak) kontrol edilir. Bağlantının kopması durumunda, çalışmakta olan GMP fonksiyonunun tanımlı timeout süresi beklenmeden **DLL_RETCODE_TIMEOUT (0xF003)** hatası döndürülür.

Bu mekanizma sayesinde, uzun timeout süresine sahip işlemler sırasında bağlantı kopması yaşanırsa, DLL ilgili timeout süresini beklemek yerine hızlı bir şekilde hata döner. Böylece bağlantı hızlıca yeniden kurulabilir ve işleme kaldığı yerden devam edilebilir.

MultipleCommand (BATCH MODE)

FP3_MultipleCommand fonksiyonu sayesinde, cihaza komutlar toplu olarak gönderilebilir ve bu sayede işlemler daha hızlı bir şekilde gerçekleştirilebilir.

Ancak Multiple Command ile gönderilen fonksiyonlarda, her bir fonksiyonun başarı durumunu ayrı ayrı kontrol etmek mümkün olmadığından hata takibi daha zor olabilir. Bu nedenle, önerdiğimiz Multiple Command kullanım yapısı aşağıdaki gibidir:

BATCH MODE

1. **START - SINGLE MODE**
2. **TICKET HEADER – OPTION FLAGS – ITEM SALE – BATCH MODE**
3. **PAYMENT – SINGLE MODE**
4. **PRINT METODLARI (total payments, print before mf, print mf) BATCH MODE**
5. **CLOSE - SINGLE MODE**

BATCH MOD'DA HATA YÖNETİMİ

Fp3_MultipleCommand fonksiyonundan dönen RETCODE değeri, yalnızca fonksiyonun batch içerisindeki komutları cihaza başarılı bir şekilde iletip ilemediğini gösterir.

Gönderdiğiniz komutlardan herhangi biri hata döndürdüyse, hata kodunu **ST_MULTIPLE_RETURN_CODE** objesinin içerisindeki **RETCODE** alanında bulabilirsiniz. Sıfırdan farklı herhangi bir değer bir hatayı işaret eder.

İlk başarısız olan komutta çalışma durur — bu komuttan sonraki komutlar çalıştırılmaz. Örneğin, Batch içerisine 5 adet prepare_ItemSale komutu gönderirseniz ve 3. komut hata dönerse, 4. ve 5. komutlar çalışmayacaktır.

```

UInt32 retcode = Json_FP3_MultipleCommand(hInt, ref hTrx, szJsonRetCodes_Out, szJsonRetCodes_Out.Length, ref IndexOfReturnCodes, &retcode);
if (retcode == 0)
{
    string retJsonString = GMP_Tools.GetStringFromBytes(szJsonRetCodes_Out);
    pReturnCodes = JsonConvert.DeserializeObject<ST_MULTIPLE_RETURN_CODE[]>(retJsonString);
    retJsonString = JsonConvert.SerializeObject(pReturnCodes);
    pstTicket = pReturnCodes[0];
    return retcode;
}
else
{
    return retcode;
}

```

Ethernet VS USB

Yapılan çalışmalar sonucunda, **Ethernet bağlantısının USB bağlantısına kıyasla daha hızlı bir iletişim sağladığı** gözlemlenmiştir.

Ayrıca cihazın Ethernet üzerinden bağlı olması, banka hostuna erişimi de hızlandıracığından, cihaz ile yazılımınız arasındaki bağlantının **Ethernet** üzerinden kurulması önerilmektedir.

Banka Host IP'leri

Cihazınız banka ile iletişim kurarken belirli **IP adreslerine** erişmeye çalışır. Bu IP'lerden bazılarının bloke olması durumunda, cihaz bağlantı denemesi yapacak ve tanımlı **timeout süresi** boyunca bekleyecektir. Bu durum, ödeme sırasında bankadan yanıt alınmasının yavaşlamasına neden olabilir.

Kontrol edilmesi gereken bir diğer önemli nokta ise **Ethernet bağlantısının sağlıklı olup olmadığıdır**. Cihaz, varsayılan ayarlarda öncelikle Ethernet üzerinden ilgili IP'lere ulaşmayı dener; bu deneme başarısız olursa **GPRS** üzerinden bağlantı kurar. Ethernet bağlantısında bir problem olması halinde işlem GPRS üzerinden başarıyla tamamlanabilir; ancak önce Ethernet denemesi yapılacağından ek bir zaman kaybı yaşanacaktır.

İzin vermeniz gereken TSM IP ve Portlarına aşağıdaki Redmine sayfası üzerinden ulaşabilirsiniz:

[Güncel TSM IP & Port Erişim Excel dosyaları - iNews \(Gmp3\) - WORLDLINE Development](#)

Hafifletilmiş Sorgular

Cihazda asılı kalan işlem (handle) kontrolü için performans kaybı oluşturan FP3_START yerine FP3_GETCURRENTHANDLE metodunun kullanılmasını; işlem takibi (Z raporu ve fiş no sorgulama) için ise FP3_ECHO metodunun tercih edilmesini performans optimizasyonu kapsamında rica ederiz.

```
void GetInterfaceHandleByIDExample(bool Exec)
{
    int start = Environment.TickCount;
    UInt32 RetCode = GMPSmartDLL.FP3_GetInterfaceHandleByID(ref Curr
    SetFunctionCallLog("FP3_GetInterfaceHandleByID", RetCode, start)
}

2 references
public void _echoTestMenuItem_Click(object sender, EventArgs e)
{
    UInt32 resp = Defines.DLL_RETCODE_RECV_BUSY;

    while (resp == Defines.DLL_RETCODE_RECV_BUSY)
    {
        ST_ECHO stEcho = new ST_ECHO();

        int start = Environment.TickCount;
        resp = Json_GMPSmartDLL.FP3_Echo(CurrentInterface, ref stEcho
        SetFunctionCallLog("FP3_Echo", resp, start);
        if (resp == 0)
            ParserClass.DisplayEcrStatus(pairingResp, stEcho);
        HandleErrorCode("FP3_Echo", resp);

        if (resp == Defines.DLL_RETCODE_RECV_BUSY)
            Thread.Sleep(1000);
    }
}
```

Property	Value
EkuNo	1
FisNo	2
Handle	0
UniquelD	{byte[24]}
ZNo	220
activeCashier	{GmpSampleSim.ST_CASHIER}
date	{GmpSampleSim.ST_DATE}
ecrMode	1
kvc	{byte[8]}
mtuSize	8192
retcode	0
status	29
szEcrNewVersion	"20000291"
szEcrVersion	"2001010000"
time	{GmpSampleSim.ST_TIME}

Unique ID ile ilgili öneri

Unique ID kullanımına ihtiyaç duyulmayan senaryolarda, FP3_START komutu gönderilirken UniqueID alanının 0xFF olarak set edilmesi gerekmektedir. Bu basit değişiklik, cihaz içerisinde gereksiz dosya oluşumunu ve buna bağlı performans kayıplarını tamamen engelleyecektir.

```
byte[] UniqueId = new byte[24] { 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF };

retcode = GMPSmartDLL.FP3_Start(CurrentInterface, ref TransactionHandle, 0, UniqueId, UniqueId.Length, null, 0, null, 0, Defines.TIMEOUT_DEFAULT);
```

Gereksiz FP3_GetTicket Kullanımı

GetTicket komutunun asıl amacı işlem esnasında bir hata ile karşılaşmanız durumunda işlemde nerede kaldığınızı görüp buna göre işlemi devam ettirmektir.

GetTicket Kullanımı ile ilgili detaylı anlatım için Olumsuz Durum Yönetimi dokümanımızı inceleyebilirsiniz.

[GMP3 olumsuz durum yönetimi. - iNews \(Gmp3\) - WORLDLINE Development](#)

Genel olarak entegratörlerimizde GetTicket fonksiyonunun yanlış yerlerde kullanıldığını görmekteyiz. Bu sebepten ötürü getTicket fonksiyonunun hatalı kullanımını bir kaç örnek ile göstermek istiyorum.

- 1- Cihaz ile bağlantı yeniden kurulduğunda cihaz üzerinde işlem olup olmadığının kontrolü için GetTicket kullanımı:
 - Bu durum için GetTicket fonksiyonu biraz ağır kaçmaktadır. Biz burada FP3_ECHO veya FP3_GETCURRENTHANDLE metodunun kullanılmasını öneriyoruz.
 - Yeni versiyonlarla beraber FP3_Echo fonksiyonunda POS dan gelen Ekü, Z ve Fiş numaralarının alınması sağlandı. Buradaki numaralar kullanılarak devam eden işlem olup olmadığı kontrol edilebilir.
- 2- Hatasız devam eden bir işlem esnasında işlemin nerede kaldığını öğrenmek için kullanım.
 - Aşağıdaki örnekteki gibi önceki işlemden herhangi bir hata kodu dönememesine rağmen GetTicket kullanımı işlemi geciktirecektir.

```
[16:15:18:072] >>>> FP3_PrintTotalsAndPayments (TO : 100000) (hInt:01611X) (hTrx:081X)
[16:15:18:801] <<<< FP3_PrintTotalsAndPayments (return:0-0x00000000:TRAN_RESULT_OK [0]:Ýplem Baýarýlý sonuýlandý. [0])
[16:15:18:821] >>>> FP3_PrintBeforeMF (TO : 100000) (hInt:01611X) (hTrx:081X)
[16:15:20:036] <<<< FP3_PrintBeforeMF (return:0-0x00000000:TRAN_RESULT_OK [0]:Ýplem Baýarýlý sonuýlandý. [0])
[16:15:20:055] >>>> FP3_GetTicket (TO : 100000) (hTrx:081X)
[16:15:20:493] <<<< FP3_GetTicket (return:0-0x00000000:TRAN_RESULT_OK [0]:Ýplem Baýarýlý sonuýlandý. [0])
[16:15:20:708] >>>> FP3_MultipleCommand (TO : 500000) (hInt:01611X) (hTrx:081X)
[16:15:21:313] <<<< FP3_MultipleCommand (return:0-0x00000000:TRAN_RESULT_OK [0]:Ýplem Baýarýlý sonuýlandý. [0])
[16:15:21:361] >>>> FP3_PrintMF (TO : 100000) (hInt:01611X) (hTrx:081X)
[16:15:29:814] <<<< FP3_PrintMF (return:0-0x00000000:TRAN_RESULT_OK [0]:Ýplem Baýarýlý sonuýlandý. [0])
[16:15:29:832] >>>> FP3_GetTicket (TO : 100000) (hTrx:081X)
[16:15:30:328] <<<< FP3_GetTicket (return:0-0x00000000:TRAN_RESULT_OK [0]:Ýplem Baýarýlý sonuýlandý. [0])
[16:15:30:364] >>>> FP3_Close (TO : 100000) (hInt:01611X) (hTrx:081X)
[16:15:30:872] <<<< FP3_Close (return:0-0x00000000:TRAN_RESULT_OK [0]:Ýplem Baýarýlý sonuýlandý. [0])
```

Entegrasyon Yazılımında Geçen Süre

Yavaşlığın nerede olduğunu görmek için GMP3 log'larını kullanabilirsiniz. GMP3 log'larında zaman kısmını açarsanız burada cihaza yolladığınız bir komutun size ne kadar sürede yanıt döndüğünü görebilirsiniz ve yavaşlığın GMP3 tarafında mı yoksa sizin kendi entegrasyonunuz da mı olduğunu anlayabilirsiniz.

GMP3 logunda, bir fonksiyonun request'i ile response'u arasında geçen süre cihazda geçen süredir. Örnek olarak aşağıda 9 saniye sürmüş bir FP3_Payment() görmekteyiz.

```
[13:03:35:772] [FUNCTION] >>>>> FP3_Payment(TO : 90000) (hInt:26DF345E) (hTrx:00004BBA0F9D9B01)
[13:03:44:254] [FUNCTION] <<<<< FP3_Payment (return:0-0x00000000:TRAN_RESULT_OK [0]:Ýplem Başarýlý sonuçlandı. [0])
```

GMP3 Logunda bir fonksiyonun response'u ile sonraki fonksiyonun request'i arasında geçen zaman sizin yazılımınızda geçen süredir.

Örnek olarak aşağıdaki logda görebileceğiniz üzere FP3_Pretotal fonksiyonundan cevap dönmesinin ardından yazılımın FP3_Payment için istek atması 19 saniye sürmüştür. Burada geçen 19 saniye GMP3'te değil yazılımın kendisinde geçen süredir. Bu konuyu kendi tarafınızda incelemeniz gerekmektedir.

```
[19:41:36:700] [FUNCTION] >>>>> FP3_Pretotal(TO : 60000) (hInt:01611X) (hTrx:081X)
[19:41:37:216] [FUNCTION] <<<<< FP3_Pretotal (return:0-0x00000000:TRAN_RESULT_OK [0]:Ýplem Başarýlý sonuçlandı. [0])
[19:41:56:591] [FUNCTION] >>>>> FP3_Payment(TO : 300000) (hInt:01611X) (hTrx:081X)
[19:42:10:154] [FUNCTION] <<<<< FP3_Payment (return:0-0x00000000:TRAN_RESULT_OK [0]:Ýplem Başarýlý sonuçlandı. [0])
```

Güncel Versiyon Kullanımı

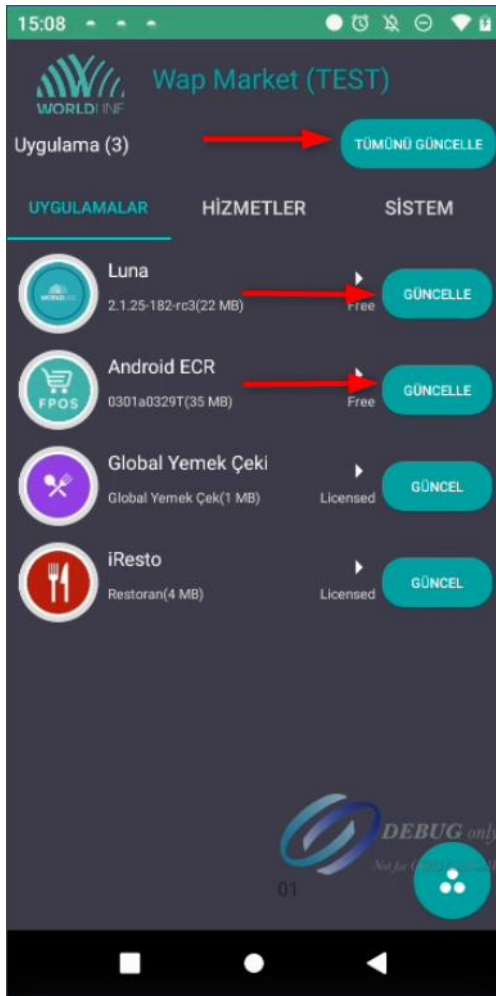
GMP3 DLL ve cihaz versiyonlarında her geçen gün iyileştirmeler ve optimizasyonlar yapılmaktadır. Siz entegratörlerimizin de bu geliştirmelerden yararlanabilmesi için cihazınızdaki uygulamaların versiyonlarını güncel tutmanızı ve yazılımınızı geliştirirken en güncel DLL versiyonlarını kullanarak güncelleme yapmanızı tavsiye ediyoruz.

DLL'in en güncel versiyonunu aşağıdaki adresten kontrol edebilirsiniz: [Files - iNews \(Gmp3\) - WORLDLINE Development](#)

GDP_v16r42.zip	16-06-2026 09:53 AM	82.7 MB	62
----------------	---------------------	---------	----

Pax 910SF için, yeni güncellemeler olup olmadığını kontrol etmek amacıyla Luna uygulamasını kullanabilir; üstteki **"Tümünü Güncelle"** butonu ile tüm güncellemeleri yapabilir veya uygulamaların sağ tarafındaki **"Güncelle"** butonlarını kullanarak tekil güncelleme yapabilirsiniz.

Diğer cihazlar, parametre indirme işlemi sırasında güncellemeleri otomatik olarak kontrol eder ve indirir. Cihaz açık olduğu sürece parametre indirme işlemi her gün otomatik olarak gerçekleştirilir.



Dijital İşyeri Nüshası

Bazı bankalar işyeri nüshasının dijitalde saklanması sağlayarak işyeri nüshasının basımını engelleyebiliyor. Bu nüshanın dijitalde alınması ile beraber fiş basım süresinden tasarruf sağlayabilirsiniz.

İşyeri nüshasının dijitalde alınması ile ilgili olarak aşağıdaki Redmine dokümanımızı inceleyebilirsiniz:

[GMP3 - Deactivating the merchant receipt copy & getting the soft copy of the merchant receipt - iNews \(Gmp3\) - WORLDLINE Development](#)

Müşteri Nüshası ile İşyeri Nüshası Ardında Geçen Süre

Müşteri Nüshası basıldıktan sonra ekranda yeşil tuşa basılmaması durumunda cihaz default olarak 10 saniye beklemekte ve ardından işyeri nüshasını basmaktadır.

Bu süre cihaz üzerinden ayarlanabilir:

Pax910SF cihazlarda → FPOS → Ayarlar → Cihaz Ayarları → Fiş Arası Süre

Diğer cihazlarda → F → YazakasaPOS ayarları → Cihaz Ayarları → Fiş Arası Süre